

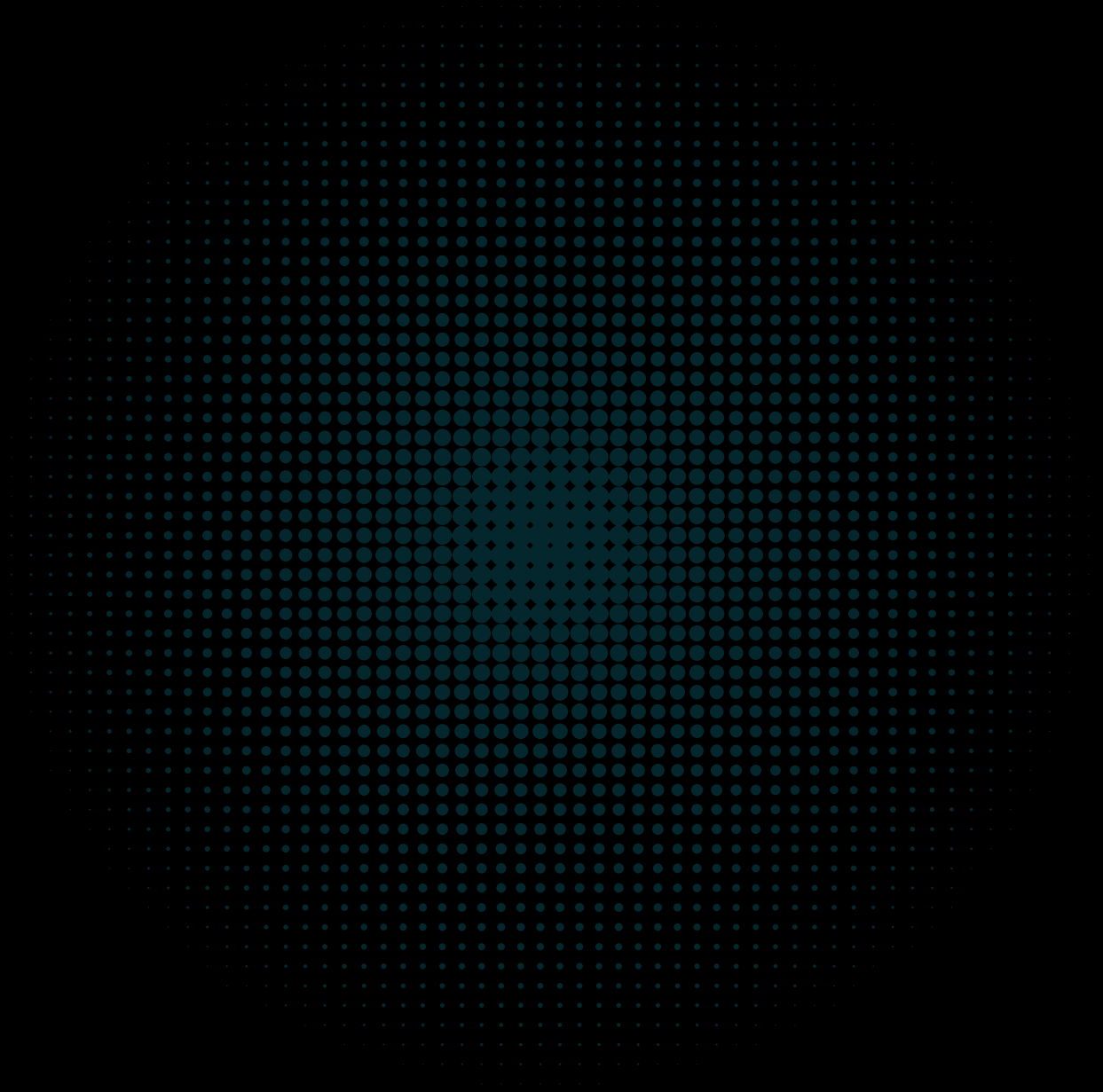
cmta.

CMTAT Cardano Specification

Functional specifications of the CMTAT on Cardano.

CMTAT Cardano version: v0.1.0

First published: August 2025



CMTAT Equivalency Assessment — `cpt-rwa-ch-de-cmta-reference` (Cardano, Swiss/German profiles)

This document assesses how much of the CMTAT security-token standard the Cardano implementation `cpt-rwa-ch-de-cmta-reference` provides.

It works through CMTA's own 54-item checklist and records, for each item, whether the behaviour exists, who may invoke it, and how it is enforced on-chain. It is written for an issuer, registrar or adviser deciding whether the codebase is a usable basis for a regulated token.

- [About CMTA and CMTAT](#) introduces the standard and its implementations;
- The [Architecture Primer](#) explains how a Solidity contract's methods map onto Cardano scripts;
- The [Annex](#) defines the vocabulary and walks through two worked examples of a refused transfer.

It is not a security audit and not a legal opinion. Read the [Warning](#) before relying on any answer below.

The code assessed, the commit it is pinned to and the toolchain are recorded under [Metadata](#). Where this codebase sits among the three assessed in this repository is set out in [Relationship to the other assessments](#).

Table of Contents

- [About CMTA and CMTAT](#)
 - [CMTA](#)
 - [CMTAT](#)
 - [Deployment variants](#)
- [Document Version](#)
- [How to Use This Document](#)
- [General Note](#)
- [Warning](#)
- [Relationship to the other assessments in this repository](#)
- [Architecture Primer \(read first\)](#)
- [CMTAT Function Equivalency Table](#)
 - [Metadata](#)
 - [Token Attributes](#)
 - [Token module](#)
 - [Pause module \(mandatory\)](#)
 - [Enforcement](#)
 - [Transfer restriction \(optional\)](#)
 - [Access Control](#)

- [Snapshot \(optional\)](#)
 - [Dividend \(optional\)](#)
 - [Credit Events \(optional\)](#)
- [Debt \(optional\)](#)
- [Guideline for New Blockchain Implementations](#)
 - [Freeze](#)
 - [CMTAT Extended](#)
 - [Forced Burn and Forced Transfer](#)
 - [Implementation Details](#)
 - [Self-Burn](#)
- [Supplementary features](#)
 - [Lifecycle and governance](#)
 - [Compliance gating](#)
 - [State integrity](#)
 - [Token metadata](#)
 - [Operational evidence](#)
- [Annex](#)
 - [Terms](#)
 - [What "validator" means here](#)
 - [Denied transfer: a sanctioned holder](#)
 - [Denied transfer: a paused protocol](#)
- [Frequently Asked Questions](#)
- [Reference](#)

About CMTA and CMTAT

CMTA

The **Capital Markets and Technology Association** (cmta.ch) is a Swiss association that gathers organizations from the financial, legal and technology sectors. CMTAT is an open standard published by CMTA: it was first developed by a working group of CMTA's members, and its development is now overseen by the Technical Committee of CMTA's Advisory Board. CMTA also publishes the fillable equivalency template that this document is a completed copy of.

CMTAT

The **CMTA Token (CMTAT)** is a security-token framework for tokenizing regulated financial instruments. It combines compliance features — conditional transfer, account freeze, token pause, forced transfer, snapshots, document attachment — with technical ones such as cross-chain transfer and upgradeability. It was initially optimized for the Swiss legal framework; CMTA states that its feature set makes it suitable for other jurisdictions too, and the framework targets equities, structured products, debt instruments and stablecoins.

CMTAT is a framework, not a codebase. CMTA documents it in a platform-agnostic way, and the equivalency template treats *the CMTAT framework*, *the CMTAT Solidity version* and *your implementation* as three distinct things — which is why every reference column in the tables below is headed "CMTAT **Solidity** corresponding feature". The Solidity version is CMTA's **reference implementation**, suitable for EVM chains, and is the one this assessment compares against; it is not the standard itself.

Implementations and specifications published or endorsed by CMTA:

Target	Language	Repository	Developed by
EVM (reference implementation)	Solidity	CMTA/CMTAT	CMTA
Tezos	SmartPy (FA2)	CMTA/CMTAT-Tezos-FA2	AirGap, with CMTA
Aztec	Noir	CMTA/private-CMTAT-aztec	Taurus, with CMTA
Solana	<i>specification only</i>	CMTA/CMTAT-Solana	Taurus, as an internal CMTA project

Cardano is not among them. The three Cardano codebases in this repository's lineage are independent implementations assessed against the framework, not official CMTA ports. That is what this document exists to evaluate.

Deployment variants

The Solidity implementation ships four deployment variants, which appear as columns in the [CMTAT Extended](#) table below. They differ in which optional modules are compiled in, not in their compliance core.

Variant	Intended instrument	What it includes
CMTAT Standard	Equities	Every feature except those specific to debt.
CMTAT Debt	Bonds and debt products	Standard, plus on-chain debt and bond attributes.
CMTAT Allowlist	Any, where transfers must stay inside a whitelist	Standard, plus an integrated allowlist rather than an external <code>RuleEngine</code> .
CMTAT Light	Stablecoins	Core only — mint, burn, address freeze, pause — without document management, snapshots or <code>forcedTransfer</code> .

The implementation assessed here does not map onto exactly one variant. It sits closest to **CMTAT Standard** — both shipped profiles target equity-style regulated securities and the debt module is out of scope (IDs 41–54) — but it also answers `y` to the **CMTAT Allowlist** row, because KYC is a built-in attestation check rather than an external rule set, *and* to the external-rules and `RuleEngine` rows, because the transfer and seizure logic scripts are themselves pluggable over the CIP-113 core. Read the variant columns as the origin of each requirement, not as a single target to match.

Document Version

Note:

- versions with the `rc` suffix are draft versions.
- version before `1.0` are also draft versions

Both numbers are below `1.0`, so by the rule above this assessment and the template it follows are each still a draft.

How to Use This Document

- Use the **CMTAT Function Equivalency Table** as the fillable assessment checklist.
- Use **Guideline for New Blockchain Implementations** as reference guidance when designing or mapping non-Solidity implementations.

General Note

- The listed functionalities are the **minimal set** required for each module.
- The key words "MUST", "MUST NOT", "REQUIRED", "SHOULD", and "MAY" in this document are to be interpreted as described in [RFC 2119](#) and [RFC 8174](#).

Warning

An implementation MAY satisfy the CMTAT standard while still failing to meet the criteria required for tokenized shares under Swiss law at the underlying-ledger level. In particular, compliance with CMTAT does not, by itself, demonstrate that decentralization-related legal criteria are satisfied.

Additional warnings specific to this implementation.

1. **Unaudited.** The repository states plainly that a formal third-party security audit is *planned and not yet completed*, and that the code should be treated as unaudited until it is. Two penetration-testing engagements were carried out by FT Labs on 23 and 26 June 2026 against commits `dd2b754` and `1beeed6`; the minting-proxy upgradability, the `GlobalStateLocation` change and the 2026-08-19/20 internal-review fixes all postdate both, so **neither report covers the assessed code**.
2. **Internal adversarial review, not an audit.** An internal review of the compliance layer found eleven defects (two critical). **Ten were fixed**, each pinned by a test in `validators/regression.ak`; the eleventh — *pause does not stop issuance*, severity medium — was **accepted as intended behaviour rather than fixed**, and is recorded here as the `Mint while pause` row of [Implementation Details](#). All eleven are written up in [documents/security/security-fixes.md](#).
3. This assessment is a **functional-mapping exercise only**. It is not a security review and not a legal opinion.
4. **Two deployment properties no on-chain code can check** (recorded by the implementation itself) materially affect several `y` answers below: the GlobalState NFT must actually land at `global_state_spend_validator`'s address at genesis, and every role credential must name something that genuinely decides (the `must_be_signed_by_credential` helper accepts a script withdrawal, so a permissive script hash in any role field fails **open**, silently). Both must be verified off-chain, after genesis and after every rotation.

Parts of this project, including this document, were written with the help of the AI coding assistant Claude Code (Anthropic).

Relationship to the other assessments in this repository

This implementation is the **successor** to the `security-token` substandard of the CIP-113 Programmable Tokens Platform (assessed in [README-cip113-security-token.md](#)), which was itself a port of [fn-bafin-cardano-sc](#). All three have now been assessed against the same template.

The table records only where the *CMTAT answers* differ; everything not listed is unchanged.

CMTAT item	<code>fn-bafin-cardano-sc</code>	CF <code>security-token</code> substandard	This implementation
ID 14 — Deactivate contract	n (partial)	n (partial)	y — irreversible <code>DeactivateContract</code> , admin-signed, requires a paused protocol first
ID 1/2/5 — Name, ticker, token id	partial	partial	y (metadata) — CIP-68 reference NFT, admin-pinned at registration
Sender-KYC toggle	fixed on	fixed on	admin-toggableable (<code>SetRequiresSenderKyc</code>)
Forced transfer while paused	blocked	blocked	allowed (deliberate; blocked only once deactivated)
Standard burn on a frozen address	n (design-dependent) — expected to fail the sender denylist check, but on a base-layer assumption that assessment could not verify	permitted (divergence)	not permitted — matches CMTAT, and now verified against the base layer; must route through the seizure path
Burn while paused	design-dependent (no transfer-hook bypass)	permitted	blocked in practice by the CIP-113 base layer (see Implementation Details)
Upgradeability	n (partial)	n (partial)	partial — swappable minting authority + registry-node upgrade + one-way <code>LockUpgrades</code>

CMTAT item	fn-bafin-cardano-sc	CF security-token substandard	This implementation
Freeze / unfreeze access control	power user <code>is_admin</code>	master admin	power user <code>is_admin</code> (delegable compliance role)

Architecture Primer (read first)

This implementation is **not** an ERC-20-style contract with named methods. It is a set of **Aiken (Plutus V3) validators** operating on Cardano's **eUTXO** ledger, following the **CIP-113 programmable-token** pattern. Understanding the following mapping conventions is required to read the tables:

- **"Validator" here means a Plutus script**, not a node or a stake pool: no network participant approves anything, and the issuer deploys these scripts themselves. The deployment compiles **ten validators across seven source modules** (`global_state`, `power_users` and `denylist` each declare both a `mint` and a `spend` validator; the other four modules declare one each), which is why the repository's parameter-application table has ten rows.
 - CIP-113 itself mandates far less — a registry node naming a minting-logic script and the transfer-logic scripts, which the platform's `dummy` substandard satisfies in a single 446-byte file. The rest is what CMTAT semantics cost on eUTXO: mutable configuration needs the GlobalState UTxO, the absence of a mapping type turns each of the two sets into a linked list (a mint plus a spend validator apiece), and upgradeability needs the proxy/authority split.
 - **One prerequisite is not in this repository**: the CIP-113 base layer (registry + programmable-logic base) must already be deployed on the target network, and `registry_policy_id` is obtained from it rather than compiled here.
- **The token is a native Cardano asset**. Its identity is the CIP-113 *issuance policy id* — pinned into every validator at compile time as `expected_issuance_policy_id` — plus a compile-time `security_asset_name: ByteArray`. There is no `name() / symbol() / decimals() / totalSupply() / balanceOf()`; those are ledger-, CIP-68- or off-chain-level concepts on Cardano.
- **Every transfer** triggers a **withdraw-0 "transfer logic" stake validator** (`transfer_logic_validator`) that must succeed for the spend to be valid — this is the CIP-113 equivalent of a CMTAT RuleEngine / transfer hook. Seizures go through a second one (`third_party_transfer_logic_validator`), and mint/burn through a third pair (`minting_logic_validator` → `minting_authority_validator`).
- **Global mutable state** lives in a single **GlobalState NFT-authenticated UTxO** (`GlobalStateDatum`, 14 fields): `transfers_paused`, `deactivated`, `mintable_amount` (remaining supply cap), `admin_credential_hash`, the two linked-list policy ids, `security_info`, the trusted-entity list, `member_root_hash`, `requires_sender_kyc`, `requires_receiver_kyc`, an immutable `network_id`, `minting_script_credential_hash`, and `upgrades_locked`.

- **A permanent proxy + a swappable authority.** `minting_logic_script.ak` is the script whose hash every CIP-113 registry node freezes forever (and from which the issuance policy id derives). It deliberately decides *almost nothing*: it authenticates `GlobalState`, refuses if `deactivated`, refuses to delegate to itself, and requires the withdraw-0 of whatever script `minting_script_credential_hash` currently names. **Every substantive mint/burn rule** — supply cap, power-user roles, KYC, denylist, destinations, registration structure — lives in `minting_authority.ak`, which the admin can replace in place via `RotateMintingScript`.
- **Three separable authorities** replace OpenZeppelin's single role registry:
 1. **Admin** — one `admin_credential_hash` in `GlobalStateDatum`, rotatable via `RotateAdmin` (dual signature: outgoing + incoming). Gates power-user list mutation, trusted entities, membership root, security info, both KYC toggles, `RotateMintingScript`, `LockUpgrades`, `UpgradeRegistryNode`, CIP-113 registration and `DeactivateContract`.
 2. **Power users** — nodes in the *power-users linked list*, each carrying independent flags: `is_admin` (add/remove denylist entries), `can_mint`, `can_burn`, `can_pause`, `can_force_transfer`. Each privileged action requires that power user's signature **and** that their node still sits at the power-users list address (so revocation actually sticks).
 3. **Trusted entities (KYC providers)** — up to 64 Ed25519 vkeys in `GlobalStateDatum.trusted_entity_vkeys` ("TEL"). They sign **off-chain** KYC attestations; they are never on-chain transactors.
- `utils.must_be_signed_by_credential` accepts **either** a transaction signature (key hash in `extra_signatories`) **or** a script withdrawal keyed by `Script(hash)` — so any authority MAY be a script (multisig / DAO). See Warning 4 for the corresponding footnote.
- **Two linked lists** (`anastasia-labs/aiken-design-patterns` v1.8.0): the **denylist** (presence of a node keyed by a 28-byte credential hash *is* the sanction; absence is proved at transfer time with a covering-node reference input) and the **power-users list**.
- **Deactivation is terminal.** `DeactivateContract` sets `deactivated = True`; a guard at the top of `global_state_spend_validator` then rejects *every* spend of the `GlobalState` UTxO, so no branch can ever unset it. Every other validator (transfer, seizure, minting proxy, minting authority, denylist, power users) independently refuses to run once the flag is set.

CMTAT Function Equivalency Table

Metadata

- Implementation assessed: [cardano-foundation/cpt-rwa-ch-de-cmta-reference](https://cardano-foundation.org/cpt-rwa-ch-de-cmta-reference) — "Programmable asset tokens on Cardano — German and Swiss profiles". The code covered is the contracts under `validators/` and `lib/`.
- Profiles: the repository ships **two profiles over one shared codebase**, a *German profile* (eWpG / BaFin) and a *Swiss profile* (CMTA Framework). Only the metadata schema and which behaviours are mandatory differ; the validators are the same.
- Implementation language: **Aiken** (on-chain, Plutus V3 / eUTXO). Off-chain tooling is out of scope for this repository (it ships contracts, `plutus.json` and a KYC test-vector script only).
- Implementation version: Aiken package `ft/rwa` **v0.0.0**; Aiken compiler **v1.1.23**; `plutus` = "v3"; repo commit `ff5624e` (2026-08-24). Deps: `aiken-lang/stdlib` v3.1.0, `anastasia-labs/aiken-design-patterns` v1.8.0, `aiken-lang/merkle-patricia-forestry` v2.1.0, `aiken-lang/fuzz` v2.2.0, `keyan-m/aiken-scott-utils` v1.4.0. CIP-113 base-layer guarantees verified against base commit `018415d`.

- Reference implementation compared against: **CMTAT v3.2.0** (Solidity) — the repository names it explicitly as its semantic reference, and also records that v3.2.0 has not itself been fully audited: **v3.0.0 is the latest fully audited CMTAT release.**

Token Attributes

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
1	Name attribute	ERC20 <code>name</code>	Public (<code>view</code>)		y (metadata)	Admin (CIP-68 metadata authority, pinned at registration)	No <code>name()</code> accessor. Two on-chain carriers: the native asset name (<code>security_asset_name</code> , a compile-time validator parameter) and the CIP-68 reference NFT minted once at registration under the same issuance policy at <code>reference_asset_name</code> (<code>((100)) label</code>). Its inline datum is <code>Constr 0 [metadata map, version, extra]</code> and holds the human-readable name. <code>reference_nft_output_is_pinned</code> forces that NFT's owner to be the GlobalState admin credential at registration, so the admin is the metadata authority by construction. Datum content is not inspected on-chain. <code>SecurityInfo.issuer_name</code> carries the issuer's legal name separately.
2	Ticker symbol attribute	ERC20 <code>symbol</code>	Public (<code>view</code>)		y (metadata)	Admin (CIP-68 metadata authority)	Same CIP-68 reference-NFT datum (<code>ticker</code>). A metadata update is the admin spending that UTxO and re-outputting it with a new inline datum — an ordinary CIP-113 owner-consent transfer, which therefore still passes the transfer logic's gates (blocked while paused and after deactivation). Not validator-enforced.
3	Reference to legally required documentation	<code>terms</code>	Public (<code>view</code>)		partial	<code>ModifySecurityInfo</code> → admin (<code>admin_credential_hash</code>)	No CMTAT <code>terms</code> (doc URI + hash) structure. <code>GlobalStateDatum.security_info</code> is opaque Data ; <code>lib/types/security/bafin.ak</code> defines the intended <code>SecurityInfo</code> shape (<code>isin</code> , <code>term_of_issue</code> , <code>crypto_security_register</code> , <code>issuer_registration</code> , <code>record_keeping</code> , <code>custodians</code> , ...) — the German profile's , and the only schema type the repository ships. The Swiss profile defines none on-chain; the field is opaque either way. Metadata only — parsed by no validator. Capped at 4 096 CBOR bytes (<code>constants.max_security_info_bytes</code>), enforced at genesis and on every update. There is no on-chain document-hash field; a URI/hash would have to be encoded inside the blob or in the CIP-68 datum.
4	No fractions	ERC20 <code>decimals</code>	Public (<code>view</code>)	- Decimals must be set to zero unless governing law permits fractions. - CMTAT Solidity allows configurable decimals at deployment	y	Ledger-enforced (integer-only)	Cardano native assets are integer-only at the ledger level ; there is no on-chain <code>decimals</code> . "No fractions" holds by construction. A CIP-68 <code>decimals</code> entry would be a display convention only and cannot create on-chain fractional balances.

For CMTAT reference implementations, decimals SHOULD be configurable rather than defaulting to zero, to support use cases beyond tokenized shares in Switzerland.

Note

On Cardano the ledger does not model decimals; a security token is intrinsically indivisible on-chain, which aligns with the Swiss "no fractions" requirement but means fractionalization (if ever needed) would require a display convention in the CIP-68 datum plus a larger issued supply. `SecurityInfo.nominal_amount` and `volume_of_issuance` document the intended economic denomination.

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
5	Token ID attribute	<code>tokenId</code>	Public (<code>view</code>)	Optional parameter.	y (metadata)	Public (ledger-derived); ISIN via admin <code>ModifySecurityInfo</code>	The unique on-chain identity is the issuance policy id — a 28-byte hash derived from the permanent minting proxy's hash, and unique per deployment because the GlobalState policy is one-shot-bound to a genesis UTxO (<code>tx0/index0</code>). Identical source therefore compiles to a different token per deployment. The regulatory identifier is <code>SecurityInfo.isin</code> . No standalone <code>tokenId</code> accessor.

For CMTAT reference implementations, `tokenId` SHOULD be included.

Note

The issuance policy id is globally unique and immutable — and, unlike in the predecessor codebases, it is now **pinned at compile time** into the transfer, seizure and minting-authority validators (`expected_issuance_policy_id`) rather than re-derived from a registry node at run time. That change closed a critical defect in which a caller could choose which token the transfer scripts policed (`security-fixes.md` §2).

Token module

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
6	Know total supply	ERC20 <code>totalSupply</code>	Public (<code>view</code>)		partial	Public (on-chain state + ledger query)	No <code>totalSupply()</code> . On-chain, <code>GlobalStateDatum.mintable_amount</code> is a remaining-mintable cap — decremented on mint, credited back on burn, and required to stay ≥ 0 by the <code>MintSecurity</code> branch. <code>SecurityInfo.volume_of_issuance</code> records the intended total. Circulating supply is the sum of on-chain UTxOs, obtainable from any chain indexer. Cap integrity rests on two additional guards: every non- <code>MintSecurity</code> branch of the GlobalState spend validator forbids a concurrent security-token mint, and the minting authority refuses any mint or burn that does not spend GlobalState.

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
7	Know balance	ERC20 <code>balanceOf</code>	Public (<code>view</code>)		partial	Public (ledger query)	No <code>balanceOf()</code> . In eUTXO a holder's balance is the sum of the security-token quantity across programmable-base UTXOs whose inline stake credential is that holder; queried off-chain via the ledger/indexer. The stake credential is the owner identity under CIP-113, and enterprise addresses (no stake credential) are rejected outright by both transfer paths.
8	Transfer tokens	ERC20 <code>transfer</code>	Token holder (<code>msg.sender</code>)		y	Token holder (owns/spends the UTXO), gated by denylist absence on both sides + KYC per flag + not-paused + not-deactivated	<code>transfer_logic_validator.withdraw</code> (<code>transfer_logic_script.ak</code>) runs on every programmable-token spend. Unique senders and unique destinations are aggregated by <code>Credential</code> (a key and a script sharing a hash count as two parties), then vetted in lockstep by <code>compliance.verify_parties</code> : denylist absence always, KYC when <code>requires_sender_kyc</code> / <code>requires_receiver_kyc</code> . Plus <code>transfers paused == False</code> and <code>deactivated == False</code> . A short action list fails closed.
9	Create tokens	<code>mint</code> / <code>batchMint</code>	Role-restricted (issuer/minter authorized)		y	Power user with <code>can_mint</code> (must sign; node must still sit at the list address)	<code>minting_logic_validator.withdraw</code> (the proxy) delegates to <code>minting_authority_validator.withdraw</code> , <code>redeemer MintBurn</code> (or <code>RegisterMint</code> for the genesis mint riding on CIP-113 registration). Requires: no registry node spent, GlobalState spent (so <code>MintSecurity</code> decrements the cap), <code>!deactivated</code> , only <code>security_asset_name</code> minted under the policy, <code>can_mint</code> + signature, and per unique destination a denylist-absence proof plus receiver KYC when <code>requires_receiver_kyc</code> . Batch = a single tx minting to many outputs (eUTXO-native). One deviation from CMTAT Allowlist is recorded in the note below.
10	Cancel tokens	<code>burn</code> / <code>batchBurn</code> / <code>burnFrom</code>	Role-restricted (issuer/burner authorized)	Implementations SHOULD use a dedicated issuer/authorized burn path for forced cancellation scenarios.	y	Power user with <code>can_burn</code> — plus <code>can_force_transfer</code> whenever the holder is sanctioned, uncooperative, or the protocol is paused	Same authority, negative <code>minted_amount</code> , gated on <code>can_burn</code> . There is no holder self-burn . Destroying existing supply <i>spends a programmable-base UTXO</i> , so the CIP-113 base layer makes this deployment's transfer logic run over it — pause gate, sender denylist proof and sender KYC included. A burn from a frozen holder, or during a pause, must therefore be routed through the seizure path (<code>third_party_transfer_logic_validator</code> , no source checks, no pause gate), which needs <code>can_force_transfer</code> as well. This is the <code>burnFrom</code> / forced-cancellation route.

Note

One deviation from CMTAT Allowlist on the mint path (ID 9), recorded in-code as **deliberate.** When a destination is the authorising `can_mint` power user's own list-authenticated credential, the receiver-KYC requirement is satisfied without an attestation. Denylist absence still applies to that operator, so a sanctioned one cannot mint to itself, and every other destination still needs a proof. Exact CMTAT Allowlist behaviour is recovered by issuing the treasury credential a KYC attestation at setup.

How a burn is actually routed, and why cancelling a sanctioned holder's position needs two roles.

Destroying supply means **spending** a programmable-base UTXO, and the CIP-113 base layer gates every such spend on the programmable-logic global's `withdraw-0`. For a burn that resolves to `TransferAct`, which requires a token-exists proof for the policy, which makes **this deployment's `transfer_logic_validator` run over the UTXO being burned**. The burn therefore inherits the whole sender-side gate: the pause flag, the sender's denylist-absence proof, and sender KYC when `requires_sender_kyc` is set. Neither base-layer escape applies: `TokenDoesNotExist` needs a registry node covering the policy and a

registered policy has none, and `UnfrackingAct` is unavailable because registration pins `unfracking_logic_script` to the empty verification key.

That leaves two routes, and which one is available depends on the holder and the pause state:

- **Standard route** — an unsanctioned, cooperative holder while transfers are unpaused. `transfer_logic_validator` passes, then `minting_authority_validator`'s `MintBurn` branch requires `can_burn` plus that operator's signature, and `GlobalState` is spent so its `MintSecurity` branch credits the burned amount back to `mintable_amount`. **Role needed:** `can_burn`.
- **Seizure route** — a sanctioned or uncooperative holder, or any burn during a pause. The transaction takes `third_party_transfer_logic_validator` instead, which performs **no source-side checks** and has **no pause gate**, requiring only `can_force_transfer`, that operator's signature, and at least one spent input carrying the security token. The minting authority still runs and still demands `can_burn`. **Roles needed:** `can_force_transfer` *and* `can_burn`.

The seizure route burns rather than moves because of what the destination scan sees: with a negative mint and no output carrying the security token, the unique-destination list is empty, so there are no destinations to vet and the seizure withdrawal is satisfied by the seized input alone.

Both routes must **spend** `GlobalState` rather than reference it, since `MintSecurity` is what enforces and restores the supply cap. Conway rejects a transaction that both spends and references the same UTXO, which is why the transfer, seizure and minting redeemers all carry a `GlobalStateLocation` instead of a fixed reference index.

Practical consequence for role assignment: grant `can_burn` and `can_force_transfer` together to whoever is expected to execute court-ordered or regulator-ordered cancellations. `can_burn` alone is not sufficient authority to retire a sanctioned holder's position. See also ID 15 (the denylist), ID 17 (forced transfer, including the all-or-nothing-per-UTxO limit) and the [Implementation Details](#) table.

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
----	-------------	--------------------------------------	---------------------------------	-------	--	--	------------------------

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
11	Approve	ERC20 <code>approve (address spender, uint256 value)</code>	Token holder	Grants a delegate permission to transfer a specific amount of tokens from the token account. This is optional, but implementations SHOULD include it since secondary market capability may depend on delegated approval to automate trading and settlement for regulated entities. Issuers SHOULD consult relevant trading and settlement venues if listing is contemplated.	n	—	Absent. eUTXO has no allowance model, and the repository lists delegated approval among the optional CMTA modules it does not implement. There is no <code>approve / transferFrom</code> analogue; delegation is expressed instead through a script credential at the holder's own address (native/multisig/Plutus), or via the admin forced transfer (ID 17).

Note

Secondary-market settlement is expressed as a single multi-party transaction (atomic swap / DvP) rather than as stored on-chain allowance. Note the practical ceiling documented by the implementation: at 25 % of the shared CIP-113 execution budget, an ordinary transfer supports ≈ 13 unique parties per side denylist-only, ≈ 9 per side with attestation KYC on both sides; the 16 KiB transaction limit binds near 40 attested parties.

Pause module (mandatory)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
12	Pause tokens	<code>pause</code>	Role-restricted (pauser/admin authorized)	Pause must prevent all transfers until <code>unpause</code> is called.	y	Power user with <code>can_pause</code> (must sign)	<code>global_state_spend_validator</code> action <code>PauseTransfers { transfers_paused: True, .. }</code> . Enforced by <code>transfer_logic_validator (transfers_paused == False)</code> . Scope, stated as a product decision: the pause covers <i>transfers</i> ; minting stays available; forced transfer (seizure) stays available; burning is blocked in practice because it spends a programmable-base UTxO and so re-enters the paused transfer logic. See Implementation Details . The pausing transaction may not move the security token itself (<code>no_security_token_spent</code>), closing the pause-and-transfer atomicity bypass.
13	Unpause tokens	<code>unpause</code>	Role-restricted (pauser/admin authorized)		y	Power user with <code>can_pause</code> (must sign)	Same <code>PauseTransfers</code> action with <code>transfers_paused: False</code> . Not available once <code>deactivated</code> — the terminal guard rejects every <code>GlobalState</code> spend, so a deactivated protocol is permanently paused.

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
14	Deactivate contract	<code>deactivateContract</code>	Role-restricted (admin authorized)	Must permanently disable the token (except in upgradeability patterns where deactivation behavior is explicitly defined).	y	Admin (<code>admin_credential_hash</code> , must sign)	<code>DeactivateContract</code> sets <code>deactivated = True</code> , admin-signed and permitted only when the protocol is already paused. Irreversible by construction, and supply need not be zero first. See the note below.

Note

Deactivation. `DeactivateContract` sets `deactivated = True`. Preconditions mirror CMTAT: already paused, admin signature, no security token moving in the same transaction, and exactly that one datum field flipped.

Irreversible by construction. A terminal guard at the top of the spend validator (`expect !input_datum.deactivated`) rejects every later spend of the GlobalState UTxO *before* branch dispatch — so no branch can clear the flag, `MintSecurity` can never run again, and the pause can never be lifted. Transfer, seizure, the minting proxy and authority, the denylist and the power-users list each refuse independently once it is set.

Supply is **not** required to be zero, so the CMTA spec's *cancelled* (burn down first) and *immobilised* (freeze in place) variants are both supported.

Enforcement

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
15	Freeze	<code>freeze</code> Or <code>setAddressFrozen(true)</code> (inferred from extracted PDF text)	Role-restricted (compliance/admin authorized)	Must block transfers to and from a given address. Single-function implementations are acceptable if they set a frozen status.	y	Power user with is_admin (must sign); the flag itself is granted/revoked by the GS admin	Implemented as a denylist (blocklist), not a per-address flag: <code>denylist.ak mint redeemer</code> <code>AddToDenylist { user_pkh, .. }</code> inserts a node into an ordered linked list keyed by the holder's 28-byte credential hash. Presence blocks both sending and receiving , matching CMTAT freeze. Two functions (add/remove) rather than one <code>setAddressFrozen</code> . Unavailable once <code>deactivated</code> . See the note below.
16	Unfreeze	<code>unfreeze</code> Or <code>setAddressFrozen(false)</code> (inferred from extracted PDF text)	Role-restricted (compliance/admin authorized)	Single-function implementations are acceptable if they clear a frozen status.	y	Power user with is_admin (must sign)	<code>RemoveFromDenylist { user_pkh, .. }</code> removes the node. Same authority and same deactivation gate.

Note

How the sanction is represented, and what it costs to check. Membership is the sanction — there is no per-address boolean anywhere. Absence is what every transfer must prove, by referencing the *covering* node whose key and link strictly bracket the party being vetted; a node keyed exactly at that hash makes both bounds fail, so the proof cannot be built at all. Keys are length-checked at insertion, so a malformed key cannot silently leave a sanction unenforceable.

- **Keyed on the bare hash**, so sanctioning a hash sanctions both the verification-key and script credential forms of it. This is the conservative direction, and the deliberate opposite of the KYC path, which distinguishes the two forms (ID 20). It is the one respect in which the denylist is broader than a CMTAT freeze, which is keyed to a single `address`.
- **No `spender` to check.** CMTAT also blocks a frozen *delegate* in `transferFrom`. There is no analogue here because there is no allowance model (ID 11): a spend is authorised by the UTxO's own credential, which is already vetted as the sender.
- **Delegated to `is_admin`** rather than the master admin, so a compliance function can be granted without handing over protocol control. Contrast the CF `security-token` substandard, where the same operation needed the master admin key.

Both transfer paths demand the absence proof for every party, which is why the denylist also blocks *receiving* and, since a burn re-enters the sender-side transfer logic, blocks an ordinary burn of that holder's tokens. The [Annex](#) traces the whole rejection.

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
17	Enforce a transfer	<code>forcedTransfer(address from, address to, uint256 value)</code>	Role-restricted (operator/compliance authorized)	Enforcement transfer is performed via <code>forcedTransfer</code> .	y	Power user with <code>can_force_transfer</code> (must sign)	<code>third_party_transfer_logic_validator.withdraw</code> . No source checks — the source may be denylisted or KYC-expired; that is the point — but every destination must be denylist-absent and, when <code>requires_receiver_kyc</code> , KYC-valid. Also requires at least one spent input actually carrying the security token, and <code>deactivated == False</code> . Deliberately NOT blocked by a pause. See the note below.
18	Partial freeze	<code>freezePartialTokens(address account, uint256 value) / unfreezePartialTokens(address account, uint256 value)</code>	Role-restricted (operator/compliance authorized)	Intended only to block a sold amount to avoid double-spend during settlement.	n	—	Absent. Denylist and pause are all-or-nothing (full address block / full global pause). There is no amount-scoped freeze. A holder can partition their own position across UTxOs, but that is a wallet convention, not an enforced hold. See the note below for why the EVM form has no direct eUTxO equivalent, and for the forced-transfer substitute available here.

Note

Forced transfer is deliberately available during a pause (ID 17) — a divergence from both predecessor codebases. Enforcement availability is judged more important than a literal reading of "prevent all transfers", following the CMTAT reference implementation, where `forcedTransfer` calls `_transfer` directly. It is still blocked once `deactivated`, and the requirement that a spent input actually carry the token stops a "seizure" being a no-op.

Operational limit on ID 17: seizure is all-or-nothing per UTxO against a sanctioned holder. The base layer returns a partial seizure's residual to the holder's own address, which then cannot produce a denylist-absence proof. Drain whole UTxOs; partial seizure is expressed by choosing which UTxOs to spend.

Why an amount-scoped freeze has no direct eUTXO form. `freezePartialTokens` works on EVM because a balance is a single integer: the contract keeps a second integer per account and checks `balanceOf(from) - frozenTokens[from] >= amount` on every transfer. Cardano has no balance. A holder's position is a set of UTxOs, each independently spendable, and a validator sees only the inputs and outputs of the transaction it is validating, never the holder's other UTxOs. "At most N of your M units may move" is therefore not expressible, because the script does not know M. The obstacle is the missing view of the account, not the arithmetic.

Three constructions recover the effect on Cardano generally, none of them implemented here:

- **UTxO partitioning.** Hold the restricted tranche at a different credential (an escrow script, an issuer-controlled sub-account) and the free tranche at the holder's own. The restriction is custody rather than a flag, so it needs the holder's consent or an issuer-side move, and no party can read a "frozen" status off the chain.
- **Per-holder account state.** An on-chain account node per holder recording total and frozen amounts, which every transfer must spend or reference. This rebuilds account semantics on eUTXO, at the cost of serialising that holder's transfers to roughly one per block, a min-ADA deposit per holder, heavier transfers, and soundness only if every unit is forced through the accounting.
- **Time-locked tranche.** The locked UTxO's validator demands a validity-range lower bound past the unlock slot. This covers vesting and lock-ups rather than discretionary compliance holds, and is not revocable early.

The settlement case is partly answered by the ledger. The template scopes this functionality to "block a sold amount to avoid double-spend during settlement". Under eUTXO the seller's UTxO is an input to the settlement transaction and cannot be spent twice, so double-*settlement* is prevented by the ledger with no freeze flag involved. What that does not prevent is reneging: the seller may still spend the UTxO in a competing transaction, invalidating the settlement rather than double-spending it. Preventing that needs escrow, which is the first construction above.

The substitute available in this implementation is a forced transfer of exactly the tranche to an issuer-controlled credential. `third_party_transfer_logic_validator` vets every token-bearing output as a destination (denylist absence always, receiver KYC when the flag is set), and the residual returning to the holder's own address is one such output, which an unsanctioned and KYC-valid holder passes. An operator holding `can_force_transfer` can therefore move N units into escrow and leave the remainder in place, reversing it later with a second forced transfer. **Ordering matters:** this fails against an already-sanctioned holder, whose residual output cannot produce a denylist-absence proof (see ID 17). Place the hold first and sanction second; the reverse order leaves only whole-UTxO seizure.

Transfer restriction (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
----	-------------	--------------------------------------	---------------------------------	-------	--	--	------------------------

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
19	Conditional transfer request	<code>RuleConditionalTransferLight.detectTransferRestriction(...)</code> and <code>approvedCount(...)</code>	Public (<code>view</code>)	Request is represented by a transfer restricted until approval count is non-zero.	n (implicit)	—	No explicit on-chain request/hold workflow. Every transfer is <i>implicitly</i> conditional: it requires denylist absence for every party and, when the corresponding flag is set, a fresh TTL-bound trusted-entity KYC attestation (or MPF membership proof). Pre-approval happens off-chain — a trusted entity signs the attestation — rather than through an on-chain pending-approval queue.
20	Conditional transfer approval	<code>RuleConditionalTransferLight.approveTransfer(...)</code>	Role-restricted (compliance/approver authorized)	Approval is consumed on transfer via <code>transferred(...)</code> ; cancellation via <code>cancelTransferApproval(...)</code> .	n (implicit)	Trusted entity (off-chain Ed25519 attestation)	The "approval" is the off-chain <code>AttestationProof</code> : a 67-byte payload <code>raw-Ed25519</code> -signed by a vkey in the TEL, binding who (<code>user_pkh</code>), which credential form (key vs script — an attestation issued for one is rejected for the other), tier , how long (<code>valid_until_ms</code> , checked against the transaction's validity-range upper bound, which must be <code>Finite</code>), which token (<code>security_policy_id</code>) and which network (<code>network_id</code>). Validated per transaction; there is no on-chain single-use consumption or cancellation. Revocation is by TEL removal, key rotation, MPF root update, or the denylist — not by cancelling an individual approval, so an attestation stays valid until its TTL.
21	Assign to whitelist	CMTAT Allowlist / Rules whitelist setters and <code>isAddressListed</code>	Role-restricted for setters; public (<code>view</code>) for checks	CMTAT Allowlist and Rules whitelist are alternative whitelist implementations.	y	Admin manages the TEL, the membership root and both toggles; trusted entities issue attestations	Two interchangeable mechanisms in <code>lib/kyc/verify.ak</code> : (a) Attestation — Ed25519 proof by a TEL issuer (<code>AddTrustedEntity</code> / <code>RemoveTrustedEntity</code> / <code>UpdateTrustedEntity</code> , admin-gated, vkeys length-checked and kept sorted without duplicates, max 64); (b) Membership — a Merkle-Patricia-Forrestry inclusion proof against <code>member_root_hash</code> (admin-updated via <code>UpdateMemberRootHash</code> ; must be empty or exactly 32 bytes), whose leaf binds credential form, hash, TTL, policy id and network identically. Both sides are independently toggleable : <code>SetRequiresSenderKyc</code> and <code>SetRequiresReceiverKyc</code> , each admin-gated.

Note

The KYC/allowlist model is **capability-based and off-chain-attested** rather than an on-chain address set: a valid, unexpired issuer signature (or MPF membership proof) is presented at transfer time. This keeps per-address on-chain state small and supports tiered KYC (`tier_user` / `tier_institutional` / `tier_vlei`; the validator currently checks only that the tier is not the `0x00` invalid sentinel). `lib/kyc/verify.ak` exports `membership_leaf_key` and `membership_leaf_value` as the **normative** encoders — the off-chain tree builder must produce byte-identical leaves. The denylist (IDs 15/16) is the complementary on-chain blacklist and is checked unconditionally, independent of both KYC flags. One receiver-side exemption is deliberate and on record: a `can_mint` power user minting to their own list-authenticated credential is not asked for an attestation (see ID 9).

Access Control

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
22	Grant role	<code>grantRole(bytes32 role, address account)</code>	Role admin (<code>DEFAULT_ADMIN_ROLE</code> or role admin)	Used for roles such as <code>ALLOWLIST_ROLE</code> , <code>DEBT_ROLE</code> , <code>OPERATOR_ROLE</code> , <code>COMPLIANCE_MANAGER_ROLE</code> .	y	Admin (<code>admin_credential_hash</code> , must sign)	Roles are power-user nodes. <code>power_users.ak mint redeemer AddPowerUser { new_power_user_key, .. }</code> inserts a <code>PowerUser { is_admin, can_mint, can_burn, can_pause, can_force_transfer }</code> . The KYC-issuer role is <code>AddTrustedEntity</code> on <code>GlobalState</code> . Both admin-gated, and both refused once <code>deactivated { active_admin_from_ref_input traps}</code> .

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
23	Revoke role	<code>revokeRole(bytes32 role, address account)</code>	Role admin (DEFAULT_ADMIN_ROLE or role admin)	AccessControl role removal.	y	Admin (admin_credential_hash, must sign)	<code>RemovePowerUser { .. }</code> and <code>RemoveTrustedEntity { vkey }</code> . Flags can also be re-scoped in place via <code>spend redeemer</code> <code>ModifyPowerUser { wanted_user_state, .. }</code> . Revocation is made to stick by a change new to this codebase: every consumer authenticates a power-user node against <code>power_user_list_script_hash</code> , i.e. requires it to still live at the list's spend address — otherwise a node NFT carried out into a private wallet would keep authenticating whatever flags its holder wrote. The admin itself is rotated via <code>RotateAdmin</code> , which requires both the outgoing and incoming admin to sign.
24	Role attribution	<code>hasRole(...)</code> / <code>getRoleAdmin(...)</code>	Public (view)	In CMTAT <code>AccessControlModule</code> , <code>DEFAULT_ADMIN_ROLE</code> is treated as having all roles.	y	Public (on-chain datum inspection)	Role membership is read on-chain by referencing a power-user node UTXO, or from <code>GlobalStateDatum.admin_credential_hash</code> ; anyone can inspect these UTXOs. No <code>hasRole</code> view function, and no single super-role : the admin gates <i>list and state governance</i> , the capability flags <i>gate operations</i> , and the admin holds none of <code>can_mint</code> / <code>can_burn</code> / <code>can_pause</code> / <code>can_force_transfer</code> implicitly.

Note

Authorization is intentionally decomposed into three authorities (see [Architecture Primer](#)) rather than a single OpenZeppelin `AccessControl` registry. Unlike in the CF `security-token` substandard, `PowerUser.is_admin` is now **live**: it is the authority that adds and removes denylist entries, deliberately split from the master admin key so a compliance function can be delegated without handing over the protocol. The corresponding hazard is recorded by the implementation itself and repeated here as Warning 4: because `must_be_signed_by_credential` accepts a script withdrawal, any of the five role credentials set to a permissive script hash is satisfiable by a stranger, and the failure is silent and fails **open**. The prescribed control is a post-grant smoke test, per role, asserting that the operation is *rejected* without that operator's signature.

Snapshot (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
25	Schedule a snapshot	<code>scheduleSnapshot(uint256 time)</code>	Role-restricted	SnapshotEngine <code>ISnapshotScheduler</code> .	n	—	No snapshot module. Listed by the repository among the optional CMTA modules deliberately not implemented.
26	Reschedule a snapshot	<code>rescheduleSnapshot(...)</code>	Role-restricted		n	—	Not implemented.
27	Unschedule a snapshot	<code>unscheduleLastSnapshot(...)</code>	Role-restricted		n	—	Not implemented.
28	Snapshot time	<code>getAllSnapshots()</code> / <code>getNextSnapshots()</code>	Public (view)		n	—	Not implemented.
29	Snapshot total supply	<code>snapshotTotalSupply(uint256 time)</code>	Public (view)		n	—	Not implemented.
30	Snapshot balance	<code>snapshotBalanceOf(uint256 time, address tokenHolder)</code>	Public (view)		n	—	Not implemented.

Note

No on-chain snapshot mechanism exists. Historical balances at a given slot can be reconstructed off-chain from the ledger — and the German profile leans on exactly that property, describing every state change as "a ledger transaction, ordered and immutable once settled" — but reconstruction is an indexer capability, not a contract feature, and nothing on-chain freezes a balance set at a point in time.

Dividend (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
31	Distribution create parameters				n	—	No distribution logic. Only the descriptive <code>SecurityInfo.dividend_payment_frequency</code> (and <code>payment_bond</code>) metadata fields exist; no create/deposit/claim validators.
32	Distribution set eligibility				n	—	Not implemented.
33	Distribution set deposit				n	—	Not implemented.
34	Distribution claim deposit				n	—	Not implemented.
35	Distribution schedule				n	—	Not implemented.
36	Distribution unschedule				n	—	Not implemented.

Note

No direct CMTAT Solidity equivalent is defined for these; they are implementation-specific (cf. the CMTA IncomeVault prototype). This codebase implements none of them on-chain — distributions are named explicitly among the optional modules left out, and would be a separate future module.

Credit Events (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
37	Flag as default	<code>setCreditEvents(...)</code> → <code>flagDefault</code>	Role-restricted	Managed in <code>ICMTATCreditEvents.CreditEvents</code> .	n	—	No credit-events structure. No <code>flagDefault/flagRedeemed/rating</code> fields in <code>GlobalStateDatum</code> . Could only be represented informally inside the opaque <code>security_info</code> blob.
38	Remove default flag	<code>setCreditEvents(...)</code> <code>flagDefault = false</code>	Role-restricted		n	—	Not implemented.
39	Flag as redeemed	<code>setCreditEvents(...)</code> → <code>flagRedeemed</code>	Role-restricted		n	—	Not implemented.
40	Set rating	<code>setCreditEvents(...)</code> → <code>rating</code>	Role-restricted		n	—	Not implemented.

Note

Credit-event state is not modeled. The admin can rewrite `security_info` (`ModifySecurityInfo`, capped at 4 096 CBOR bytes), which could carry an ad-hoc status, but there is no typed or enforced credit-event field. The nearest *enforced* terminal state is `DeactivateContract` (ID 14), which is closer to redemption/decommissioning than to a default flag.

Debt (optional)

ID	Attribute	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
41	Guarantor identifier	<code>debt().debtIdentifier.guarantor</code>	Read public; write role-restricted	Debt module.	n	—	No debt module — the repository names debt terms among the optional CMTA modules not implemented. <code>SecurityInfo</code> is an <i>equity/eWpG</i> metadata structure. No guarantor field.
42	Debtholder representative identifier	<code>debt().debtIdentifier.debtholder</code>	Read public; write role-restricted		n	—	Not present.

ID	Attribute	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
43	Unique identifier / hash	<code>tokenId()</code> and <code>terms().doc.documentHash</code>	Public (view)		partial	Public / admin (ModifySecurityInfo)	Unique id = issuance policy id + <code>SecurityInfo.isin</code> + <code>crypto_security_register</code> . No document-hash field.
44	Issuance date	<code>debt().debtInstrument.issuanceDate</code>	Read public; write role-restricted		partial	Admin (ModifySecurityInfo)	<code>SecurityInfo.term_of_issue</code> (free-form <code>ByteArray</code>) can encode issuance terms/date; not a typed date, not validator-enforced. The <i>ledger</i> records the genesis transaction's slot, which is the authoritative on-chain issuance moment.
45	Currency of payments	<code>debt().debtInstrument.currency</code>	Read public; write role-restricted		n	—	No currency field (<code>payment_bond</code> is unrelated free-form metadata).
46	Par value	<code>debt().debtInstrument.parValue</code>	Read public; write role-restricted		partial	Admin (ModifySecurityInfo)	<code>SecurityInfo.nominal_amount: Int</code> is the nominal/par value; metadata only, non-enforcing.
47	Minimum denomination	<code>debt().debtInstrument.minimumDenomination</code>	Read public; write role-restricted		n	—	Not present.
48	Maturity date	<code>debt().debtInstrument.maturityDate</code>	Read public; write role-restricted		n	—	Not present.
49	Interest rate	<code>debt().debtInstrument.interestRate</code>	Read public; write role-restricted		n	—	Not present.
50	Coupon payment frequency	<code>debt().debtInstrument.couponPaymentFrequency</code>	Read public; write role-restricted		partial	Admin (ModifySecurityInfo)	Only <code>SecurityInfo.dividend_payment_frequency</code> (a <i>dividend</i> , not coupon, frequency) exists as free-form metadata.
51	Interest schedule format	<code>debt().debtInstrument.interestScheduleFormat</code>	Read public; write role-restricted		n	—	Not present.
52	Interest payment date	<code>debt().debtInstrument.interestPaymentDate</code>	Read public; write role-restricted		n	—	Not present.
53	Day count convention	<code>debt().debtInstrument.dayCountConvention</code>	Read public; write role-restricted		n	—	Not present.
54	Business day convention	<code>debt().debtInstrument.businessDayConvention</code>	Read public; write role-restricted		n	—	Not present.

Note

Both shipped profiles target **equity-style regulated securities** (eWpG / Swiss CO ledger-based securities), so the CMTAT *debt* module is out of scope. The `SecurityInfo` type instead captures equity attributes — `isin`, `security_type`, `security_form`, `security_class`, `record_keeping`, `entry_type`, `without_voting_rights`, `transfer_restrictions`, `third_party_rights`, `company_consent`, `participate_settlement`, `nominal_amount`, `volume_of_issuance`, `issuer_name`, `issuer_registration`, `crypto_security_register`, `share_custodian`, `token_custodian`. All of it is **non-enforcing**: the field is opaque `Data` in the datum, parsed by no validator, and the repository states explicitly that populating it correctly and keeping it consistent with the register is an off-chain duty of the issuer and registrar. A debt profile would need a second `SecurityInfo` schema, which the shared-codebase design accommodates without touching the validators.

Guideline for New Blockchain Implementations

Freeze

To be compatible with [ERC-3643](#), freeze is implemented with a single function:

```
setAddressFrozen(targetAddress, frozenStatus).
```

For non-EVM blockchains, implementations MAY separate this into two distinct functions:

```
freeze(address targetAddress)
unfreeze(address targetAddress)
```

Note

This implementation uses the **two-function** form via the denylist mint validator:

`AddToDenylist { user_pkh, .. }` and `RemoveFromDenylist { user_pkh, .. }`, both gated on a power user holding `is_admin` plus their signature. "Frozen status" is **presence of a node** in an ordered linked list keyed by the 28-byte credential hash; absence is proven at transfer time by a *covering-node* reference input.

Semantics match CMTAT's freeze rather than exceeding it. CMTAT v3.2.0 is not send-side only: `ValidationModule._canTransferIsFrozen` blocks a standard transfer if `spender`, `from` **or** `to` is frozen, `_canMintBurnByModule` refuses to mint *to* a frozen address, and refuses to burn *from* one. A denylisted address here can likewise **neither send nor receive**, which means:

- forced transfers *to* a denylisted destination are blocked (though not *from* one — see ID 17);
- an ordinary **burn** of that holder's tokens is blocked too, since a burn re-enters the sender-side transfer logic.

Worth knowing when building transactions:

- the denylist keys on the **bare hash**, so sanctioning a hash sanctions both the verification-key and script credential forms — the conservative direction, where the KYC path deliberately distinguishes them;
- the covering node is authenticated once per *run* of adjacent parties citing it, so keep parties that share a node adjacent in the action lists.

CMTAT Extended

In the table below, the CMTAT framework extended features are mapped to Solidity features.

CMTAT Functionalities	CMTAT Solidity corresponding features	CMTAT Allowlist	CMTAT Light	CMTAT Debt	CMTAT Standard	Present in implementation being approved (y/n)	Implementation details
On-chain snapshot	<code>snapshotModule</code> and <code>snapshotEngine</code>	✓	✗	✓	✓	n	Not implemented (see IDs 25–30).
Forced transfer	<code>forcedTransfer</code>	✓	✗	✓	✓	y	<code>third_party_transfer_logic_validator</code> — power user <code>can_force_transfer</code> ; destinations denylist-absent (+ KYC per flag); source unchecked; not blocked by pause; blocked once deactivated; must actually seize at least one token-bearing input.

CMTAT Functionalities	CMTAT Solidity corresponding features	CMTAT Allowlist	CMTAT Light	CMTAT Debt	CMTAT Standard	Present in implementation being approved (y/n)	Implementation details
Forced burn	<code>forcedBurn</code>	✗	✓	✗	✗	y	Negative mint through <code>minting_authority_validator</code> , gated on <code>can_burn</code> . No holder consent is needed, but a burn is not exempt from the compliance gates: it spends a programmable-base UTxO, so the sender-side transfer logic runs over it. Retiring a sanctioned or uncooperative holder's position therefore needs the seizure path too, i.e. <code>can_burn</code> and <code>can_force_transfer</code> . See Forced Burn and Forced Transfer .
Freeze partial token	<code>freezePartialTokens</code> / <code>unfreezePartialTokens</code>	✓	✗	✓	✓	n	Not implemented (see ID 18).
Integrated whitelisting/allowlisting	CMTAT Allowlist	✓	✗	✗	✗	y	Built-in KYC allowlist: trusted-entity attestations + MPF membership tree, independently toggleable per side, plus the integrated denylist. See IDs 15/16, 21.
External whitelisting/allowlisting	CMTAT with rule whitelist	✗	✗	✓	✓	y	KYC is attested by external trusted entities (off-chain Ed25519 issuers registered in the TEL). The transfer and seizure logic scripts are themselves externally pluggable rule sets over the CIP-113 core, re-pointable via <code>UpgradeRegistryNode</code> .
RuleEngine / transfer hook	CMTAT with RuleEngine	✗	✗	✓	✓	y	The CIP-113 model is a transfer hook: <code>transfer_logic_validator</code> (withdraw-0) runs on every programmable-token spend and is this deployment's RuleEngine equivalent; <code>third_party_transfer_logic_validator</code> and the minting proxy/authority pair are the seizure and issuance equivalents.
Upgradeability	CMTAT Upgradeable version	✓	✓	✓	✓	partial	New in this codebase; neither predecessor had any upgrade path at all. No <code>delegatecall</code> exists on Cardano, but two in-place paths do: the minting authority , swappable by the admin via <code>RotateMintingScript</code> ; and the transfer and seizure logic , re-pointable through the CIP-113 registry node via <code>UpgradeRegistryNode</code> . <code>LockUpgrades</code> closes both permanently. Immutable throughout: the minting proxy, <code>global_state_spend_validator</code> , and the two linked-list validators. See the note below.
Fee payer / gasless	CMTAT with ERC-2771 module	✓	✗	✗	✓	n	No ERC-2771 meta-transaction module. Fee sponsorship is achievable off-chain (a sponsor supplies fee and collateral inputs to the user's transaction) but is not codified on-chain.

Note

Upgradeability here is a *proxy-and-registry* model rather than an EVM storage proxy. The two paths differ in what they touch:

- **The minting authority.** `minting_logic_script.ak` is a deliberately near-empty permanent proxy — the script every CIP-113 registry node freezes forever — that delegates to whatever `GlobalStateDatum.minting_script_credential_hash` names. `RotateMintingScript` is **single-signature by design**, so a compromised authority need not co-sign its own replacement.
- **The transfer and seizure logic.** Re-pointed through the registry node via `UpgradeRegistryNode`, admin-signed, with this deployment's own invariants re-asserted on the way through: `global_state_cs` pinned, unfracking kept forbidden, and both fields kept `Script` credentials.

The implementation states the trade-off explicitly: because the permanent proxy checks nothing beyond "the named authority ran", a replacement authority inherits the full burden of correctness.

Five invariants are therefore enforced in the authority or nowhere:

- no registry node spent during a supply change;
- GlobalState must be spent;
- only `security_asset_name` mintable;
- not deactivated;
- registration structure.

`LockUpgrades` is the on-chain form of "this token's rules can no longer change" — meaningful to holders and regulators because an admin-key compromise cannot undo it — at the cost of also giving up the ability to patch a buggy authority.

One nuance the implementation records rather than blocks. Because

`UpgradeRegistryNode` may read GlobalState as a *spent* input, an admin can pair a final upgrade with `LockUpgrades` (or with `RotateAdmin`, or with `DeactivateContract`) in the same transaction and have it see the pre-state. It grants no power the admin did not already have across two transactions, but "locked at slot N" implies "no upgrade *after* N", not "none *at* N".

Two unrelated rows of the table above: **gasless/sponsorship** stays off-chain, and the KYC proof design is **network-bound** (`network_id`, immutable after genesis) to prevent cross-network attestation replay.

Forced Burn and Forced Transfer

In the standard burn function, tokens from a frozen wallet MUST NOT be burnable. CMTAT offers `forcedTransfer` to force a transfer or a burn.

If `forcedTransfer` is not available, implementations MAY implement only `forcedBurn` (as in CMTAT Light). Implementations MAY also implement both. In that case, only `forcedBurn` SHOULD burn tokens, and `forcedTransfer` SHOULD NOT burn tokens.

With the CMTAT Solidity version, when `forcedTransfer` is available, `forcedBurn` is not implemented to reduce contract code size. This limitation MAY not apply to other blockchains.

Note

This implementation provides **both**, and satisfies the "tokens from a frozen wallet MUST NOT be burnable in the standard burn function" requirement — which the CF `security-token` substandard did not, because it short-circuited the transfer hook for mint and burn outright, and which `fn-bafin-cardano-sc` was only expected to, on an assumption about base-layer behaviour its assessment could not verify. No new check was added. Destroying existing supply spends a programmable-base UTxO, and the CIP-113 base layer gates every such spend on this deployment's transfer logic (via `TransferAct`), which requires a sender-side denylist-absence proof, sender KYC where enabled, and an unpaused protocol. Neither base-layer escape applies — `TokenDoesNotExist` needs a registry node covering the policy and a registered policy has none, and `UnfrackingAct` is unavailable because registration pins `unfracking_logic_script` to the empty vkey.

The consequence is that the two CMTAT paths are **not** cleanly separated the way the guideline suggests. A burn of an *unsanctioned*, *cooperative* holder's tokens while unpaused is a pure `forcedBurn` (`can_burn`, no destination, does burn). A burn of a *sanctioned* or *uncooperative* holder's tokens, or any burn during a pause, must be carried on the seizure path — so `forcedTransfer` **does** burn in that shape, and requires `can_force_transfer` in addition to `can_burn`. Grant both roles together to whoever is expected to execute court-

or regulator-ordered cancellations. Code size is not a binding constraint on Cardano, so keeping the two validators separate remains intentional.

Implementation Details

Functionalities	CMTAT Solidity	Access Control (CMTAT Solidity)	Note	Present in implementation being approved (y/n)	Access Control (implementation being approved)	Implementation details
Mint while pause	✓	Role-restricted (minter/issuer authorized)	Dedicated cross-chain mint cannot be performed while paused.	y	Power user <code>can_mint</code>	Allowed by explicit decision. <code>verify_mint_or_burn</code> reads <code>deactivated</code> but never <code>transfers_paused</code> , and a fresh mint spends no programmable-base UTxO, so the transfer logic never runs over it. Pinned by the test <code>mint_succeeds_while_transfers_are_paused_by_design</code> . See the note below.
Burn while pause	✓	Role-restricted (burner/issuer authorized)	Dedicated cross-chain burn cannot be performed while paused.	n (diverges)	Power user <code>can_burn</code> + <code>can_force_transfer</code>	Blocked in practice. The burn path never reads the pause flag, but a burn spends a programmable-base UTxO, so the base layer runs this deployment's paused transfer logic over it. The only route during a pause is the seizure path — hence the extra <code>can_force_transfer</code> . See the note below.
Self-Burn for everyone	✗	Not permitted	Token holders cannot burn their own tokens; only authorized addresses can burn.	n (not permitted)	—	Matches CMTAT: holders cannot self-burn. Every burn requires a <code>can_burn</code> power-user signature.
Self-Burn for authorized addresses	✓	Role-restricted (authorized burner)		y	Power user <code>can_burn</code>	Authorized burn via <code>minting_authority_validator</code> negative mint, with <code>GlobalState</code> spent so the cap is credited back.
Standard burn on a frozen address	✗	Not permitted in standard burn path	Requires <code>forcedTransfer</code> or <code>forcedBurn</code> .	n (not permitted)	—	Matches CMTAT. The CF <code>security-token</code> substandard permitted it, a divergence recorded in its own assessment; <code>fn-bafin-cardano-sc</code> was expected to match CMTAT here but could not confirm it. A sanctioned holder cannot produce the denylist-absence proof the burn's sender-side transfer logic demands, so the standard burn path fails; the seizure path is the required route.
Burn tokens with <code>forcedTransfer</code>	✓	Role-restricted (operator/compliance authorized)	See notes above.	y (required in some cases)	Power user <code>can_force_transfer</code> + <code>can_burn</code>	The seizure withdrawal only requires that at least one <i>input</i> carries the security token; a transaction with no token-bearing outputs has no destinations to vet, so combining it with a negative mint burns the seized position. This is the only route to retire a sanctioned holder's tokens, and it diverges from the guideline's "only <code>forcedBurn</code> SHOULD burn".

Note

One rule decides all three pause rows above: whether the operation *spends* a programmable-base UTxO carrying the security token. Only then does the CIP-113 base layer dispatch to a transfer-logic script, and only then can a pause apply. A mint spends none, so it proceeds; a burn spends one, so it does not; a seizure is dispatched to the path that has no pause gate. The [Annex](#) traces this.

Mint while paused follows the CMTAT reference implementation, where `mint` goes through `_update` rather than the pause-checked transfer path. Two consequences are accepted and documented rather than fixed — this is the one internal-review finding closed by decision instead of by a change:

- position sizes can change while the register is otherwise static, though every change is still signed, on-chain, role-gated and cap-enforced;
- a holder minted to mid-pause cannot move the tokens until the pause lifts, so off-chain procedure must not mint to third parties during a pause.

Burn while paused corrects the pause exemption as originally documented; see [security-fixes.md](#), *Burning needs more authority than `can_burn`*.

Self-Burn

Only the issuer and authorized addresses (not the token holder) can burn a token in CMTAT Solidity, which reflects legal requirements in several jurisdictions.

Once issued, a security can only be cancelled by its issuer, not its holder. Since the token represents the security, the same rule applies. An investor who wants to exit should transfer to the issuer, who can then cancel when legally permitted.

You MAY still add self-burn in your version if it fits your legal or business context.

Note

This implementation **follows the CMTAT legal model**: token holders cannot self-burn. Cancellation is a power-user (`can_burn`) operation only. An exiting investor transfers to an authorized address — itself denylist- and KYC-gated — which then burns. Note that the burning transaction must still clear the sender-side gates for whoever currently holds the tokens, so a cooperative exit is a two-step flow (transfer to treasury, then burn from treasury) rather than a burn directly out of the investor's UTxO.

Supplementary features

Features present in this implementation beyond the CMTAT baseline, grouped by what they govern.

Lifecycle and governance

The token has a defined end state and two bounded upgrade paths, each closable on purpose.

- **Irreversible decommissioning switch** (`DeactivateContract`) with a terminal guard that makes the GlobalState UTxO permanently unspendable, satisfying CMTAT ID 14 and supporting both the *cancelled* and *immobilised* readings of the CMTA spec.
- **One-way upgrade lock** (`LockUpgrades`) — an on-chain, admin-compromise-proof commitment that the token's mint and transfer rules are final.
- **Swappable minting authority behind a permanent proxy** (`RotateMintingScript`) — an eUTXO analogue of a proxy upgrade, with the five invariants any replacement must preserve documented in-code.
- **Governed transfer-logic upgrade** (`UpgradeRegistryNode`) that re-asserts this deployment's own invariants over the base layer's freer registry semantics.
- **Rotatable admin with dual-signature handover** (`RotateAdmin`).

Compliance gating

Who may hold and move the token is decided by off-chain attestations checked on-chain, with each side of a transfer independently controllable.

- **Independently toggleable sender and receiver KYC** (`SetRequiresSenderKyc` / `SetRequiresReceiverKyc`).
- **Tiered, TTL-bound, off-chain KYC attestations** (`tier_user` / `tier_institutional` / `tier_vlei`), verified on-chain via Ed25519 or an MPF membership proof, **network-bound** and **policy-bound** to prevent replay, with the TTL checked against the transaction's validity-range upper bound (a non-`Finite` bound is rejected).
- **Credential-form binding**: a KYC attestation names whether it was issued for a verification key or a script, and is rejected for the other form — closing a holder-identification defect under eWpG §14. The denylist deliberately goes the other way, sanctioning both forms of a hash.

State integrity

Supply, roles and the GlobalState datum are constrained at genesis and on every mutation, so no single transaction can leave the protocol in a state its rules did not authorise.

- **On-chain supply cap** (`mintable_amount`) enforced atomically with each mint, backed by two structural guards: non-`MintSecurity` branches forbid concurrent security-token mints, and admin actions may not move the security token at all in the same transaction (closing pre-state atomicity bypasses).
- **Revocation that sticks**: every power-user node read is pinned to the list's spend address, so a node NFT removed from the list cannot keep authenticating stale role flags.
- **Datum growth caps** (≤ 64 trusted entities, $\leq 4\,096$ B `security_info`, ≤ 512 B per entity metadata) enforced at genesis and on every mutation.
- **Genesis sanitisation** of the initial datum: born unpaused, born unlocked, born active, 28-byte credential hashes, in-range `network_id`, well-formed membership root, sorted duplicate-free TEL, and two *distinct* linked-list policy ids both minted in the genesis transaction.
- **Griefing-hardened credential registration**: all four withdraw-0 validators whitelist `RegisterCredential` only, so a third party cannot de-register a stake credential and brick the protocol.

Token metadata

Two carriers, neither parsed by any validator.

- **CIP-68 reference NFT** for token metadata, minted once at registration and pinned to the admin credential.
- `SecurityInfo` **regulatory metadata** block for eWpG/German and Swiss disclosure.

Operational evidence

What the repository measures and pins, as distinct from what it enforces.

- **Measured execution budget**: `aiken bench` scaling benchmarks for the transfer and seizure paths, published per-party costs, and conservative per-transaction party maxima at 25 % of the shared CIP-113 budget.
- **Regression suite** (`validators/regression.ak`) pinning every fixed defect, with positive controls paired to each negative test.
- **eUTXO-native batch mint/transfer** — a single transaction can create or move tokens across many holders, subject to the budget limits above.

Annex

Terms

Cardano concepts a reader coming from Solidity needs in order to follow the tables above. Terms are defined as this implementation uses them, not in full generality.

Term	Definition
------	------------

Term	Definition
eUTXO	Cardano's ledger model. There are no accounts and no contract storage: value lives in discrete <i>unspent transaction outputs</i> , and a transaction consumes some and creates others. A holder's balance is a set of UTxOs, not an integer — which is why <code>balanceOf</code> , <code>totalSupply</code> and amount-scoped freezes have no direct form here (IDs 6, 7, 18).
UTxO	One unspent output: an address, a value (ADA plus any native assets), and optionally a datum. Spending it requires satisfying its address's credential.
Native asset	A token the ledger tracks natively, identified by a policy id (28-byte hash of its minting script) plus an asset name . No token contract exists; the ledger enforces conservation. Quantities are integers, which is what makes "no fractions" (ID 4) hold by construction.
Minting policy	The script that authorises creating or destroying units of its own policy id. A negative mint is a burn.
Validator / Plutus script	A pure on-chain predicate that either accepts or rejects the transaction spending, minting, or withdrawing under it. It is <i>not</i> a node, a validator stake pool, or a network participant. See What "validator" means here .
Plutus V3	The script interpreter version these contracts compile to. Aiken is the source language; <code>aiken.toml</code> pins the compiler.
Datum	Data attached to a UTxO. An inline datum stores it in the output directly. This is the only mutable state available: changing it means spending the UTxO and re-creating it with new contents.
Redeemer	Data the spender supplies with the transaction, naming which branch of a validator to take and where to find things. It is an argument, never an authority — a redeemer claim must be checked against the transaction.
Credential	Either a <code>VerificationKey(hash)</code> or a <code>Script(hash)</code> . An address carries a payment credential and, usually, a stake credential . Under CIP-113 the <i>stake</i> credential is the holder identity, which is why an enterprise address — one with no stake credential — is rejected by both transfer paths.
Reference input	A UTxO a transaction reads without spending. Used here to prove denylist absence and to read power-user roles. Conway forbids a UTxO being both spent and referenced in one transaction, which is why several redeemers carry a <code>GlobalStateLocation</code> instead of a fixed index.
Withdraw-0	A zero-ADA reward withdrawal included purely to force a script to run once per transaction. This is how CIP-113 invokes transfer logic: it is the eUTXO equivalent of a CMTAT transfer hook or RuleEngine call.
State thread / NFT-authenticated UTxO	A UTxO made unique and unforgeable by holding a one-of-a-kind token. <code>GlobalStateDatum</code> is authenticated this way — finding the NFT <i>is</i> the proof the datum is genuine.

Term	Definition
On-chain linked list	eUTxO has no mapping type, so a set is built as ordered nodes, each a UTxO keyed by a hash and linking to the next. Membership is presence of a node; absence is proved by referencing the <i>covering</i> node whose key and link strictly bracket the target. Used for the denylist and the power-users list.
Node	One element of such a list: a single UTxO sitting at the list's own spend address, holding exactly ADA plus one NFT of the list's policy, whose asset name is the 4-byte prefix <code>"Node"</code> followed by the 28-byte key. The key is fixed by the NFT and cannot be rewritten; the link and payload live in the mutable datum , which is why every reader also pins the node's address. Not a <code>cardano-node</code> , and distinct from a CIP-113 registry node , which is an element of the base layer's own list.
MPF (Merkle Patricia Forestry)	A hash tree whose root fits in one datum field, letting membership be proved by an inclusion proof instead of an on-chain node per member. The alternative KYC mechanism to signed attestations.
Ed25519	The signature scheme used both by the ledger for transaction signatures and, here, by trusted entities signing off-chain KYC attestations verified inside the validator.
Validity range	The slot window in which a transaction may be accepted. A TTL is enforced by requiring the range's upper bound to be finite and no later than the attestation's expiry — an on-chain script cannot read "now".
Execution units	The CPU and memory budget a transaction's scripts share (10 000 M CPU / 14 M memory). Memory binds first here, which is what caps parties per transfer.
CIP-68	The metadata standard where a reference NFT holds a datum describing the token. This implementation's name and ticker (IDs 1, 2) live there.
CIP-113	The programmable-token proposal this codebase targets. A registry node names the minting-logic and transfer-logic scripts for a policy; the programmable-logic base is the script address every such token must sit at, so that no transfer escapes its logic. The base layer is a deployment prerequisite and is not in this repository.

What "validator" means here

The word collides with Cardano's other meaning of it. Throughout this document a **validator is a Plutus script**, never a stake pool, a block producer or any other network participant. No one votes on a transaction, and the issuer deploys these scripts themselves.

A validator is a pure predicate compiled to UPLC. It answers one yes/no question about the transaction being submitted, and it sees only that transaction: inputs, outputs, the mint field, withdrawals, signatories, the validity range, its own redeemer and datum. There is no storage between runs, no call to another contract, no clock, and no view of any UTxO the transaction does

not itself reference. Plutus phase-2 validation is all-or-nothing — if any script in a transaction fails, the whole transaction fails — which is what makes withdraw-0 delegation sound.

Its **hash is its identity**. For a spend the hash is the address; for a mint it is the policy id; for a withdrawal it is the stake credential. Change one byte of source and the hash changes, and with it the address or the token. `minting_logic_script.ak` records a real instance: routing one lookup through a shared helper moved its hash from `6defc595...` to `b6e4e07a...` with identical behaviour, and because the issuance policy id derives from that hash, the change would have orphaned every token already issued.

Which question a validator answers depends on its **script purpose**. Aiken declares one handler per purpose, and this deployment uses four:

Purpose	The question it answers	Used here for
<code>spend</code>	May this UTxO at my address be consumed?	GlobalState; the denylist and power-users list nodes
<code>mint</code>	May tokens under my policy id be created or destroyed?	The GlobalState NFT; the two list-node policies
<code>withdraw</code>	May a reward withdrawal from my stake credential happen?	Transfer logic, seizure logic, the minting proxy and the minting authority — the withdraw-0 hook
<code>publish</code>	May this certificate be posted?	Refusing every certificate except <code>RegisterCredential</code> , so no third party can de-register a credential and brick the protocol

The [Architecture Primer](#) lists the ten validators this deployment compiles. Their handlers divide as follows:

Module	Validators (handler)
<code>global_state</code>	<code>global_state_mint_validator</code> (mint), <code>global_state_spend_validator</code> (spend)
<code>power_users</code>	<code>mint</code> (mint), <code>power_users_validator</code> (spend)
<code>denylist</code>	<code>mint</code> (mint), <code>denylist_validator</code> (spend)
<code>minting_logic_script</code>	<code>minting_logic_validator</code> (withdraw, publish)
<code>minting_authority</code>	<code>minting_authority_validator</code> (withdraw, publish)
<code>transfer_logic_script</code>	<code>transfer_logic_validator</code> (withdraw, publish)
<code>third_party_transfer_logic_script</code>	<code>third_party_transfer_logic_validator</code> (withdraw, publish)

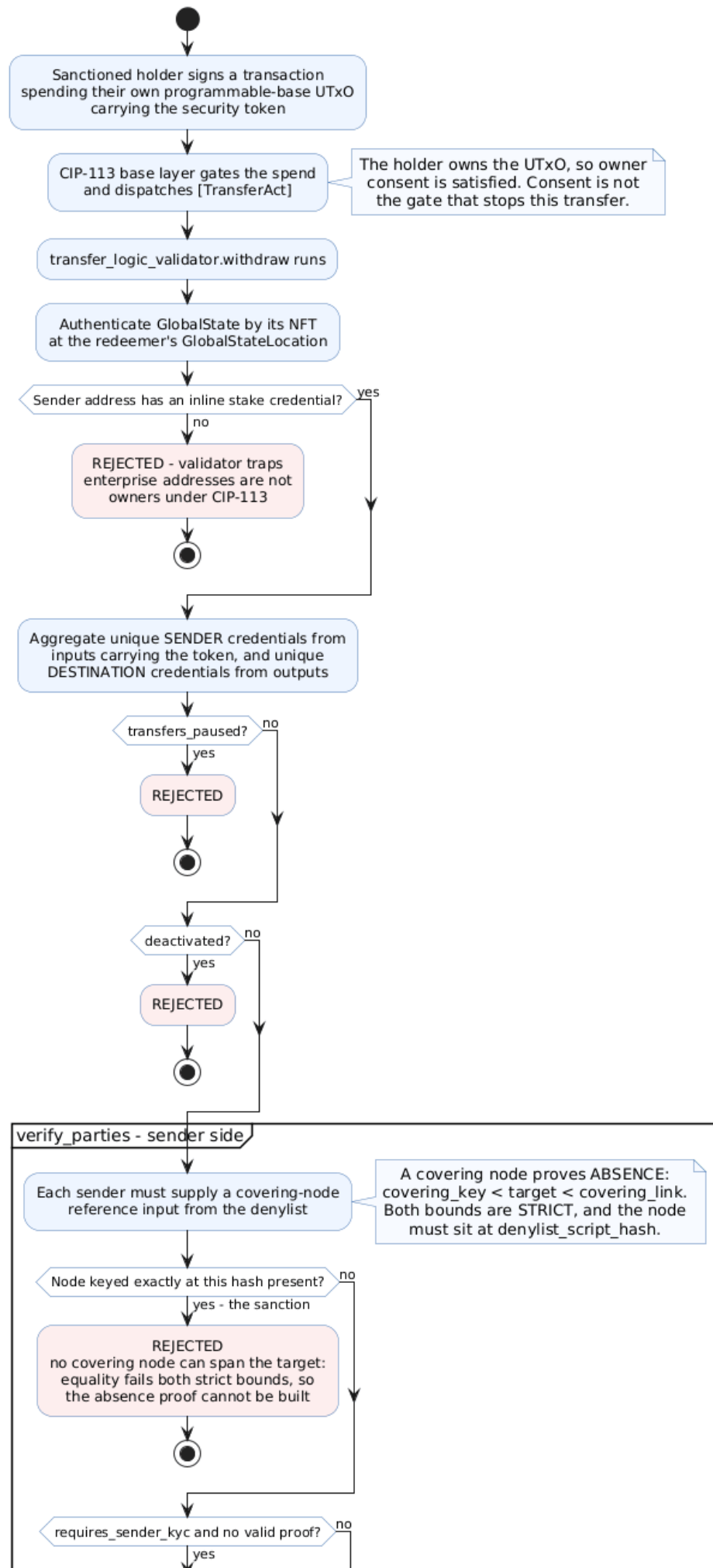
Validators are immutable once deployed. A Plutus script cannot be patched. Every upgrade path here — `RotateMintingScript`, `UpgradeRegistryNode` — is therefore indirection through mutable *state* that names a different script hash, never modification of a script. That is also why `LockUpgrades`, which freezes the indirection, is equivalent to freezing the rules.

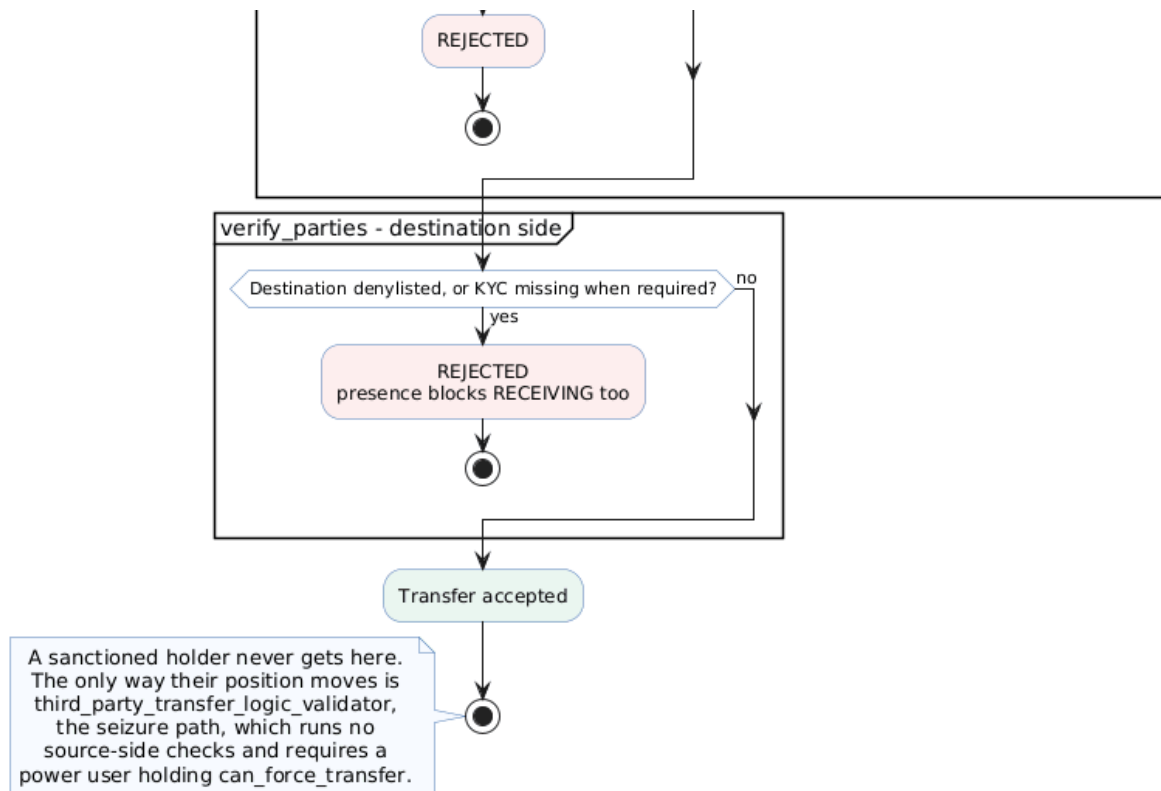
Parameters are applied at compile time, and each parameterisation yields a different hash. Hence the repository's ten-row parameter-application table and its strict compile order: the `GlobalState` policy id must exist before the proxy can be compiled, the proxy's hash yields the issuance policy id, and that id is then baked into the transfer, seizure and minting-authority validators as `expected_issuance_policy_id`.

Denied transfer: a sanctioned holder

The denylist is this implementation's freeze (IDs 15, 16). Sanction is *presence* of a node in the ordered list keyed by the holder's 28-byte credential hash; every transfer proves *absence* by referencing a covering node whose key and link strictly bracket the party. When a node keyed exactly at that hash exists, both strict bounds fail, so no covering node can be produced and the proof simply cannot be built. Nothing needs to detect the sanction — the transaction is unconstructible.

A sanctioned holder tries to move their own tokens

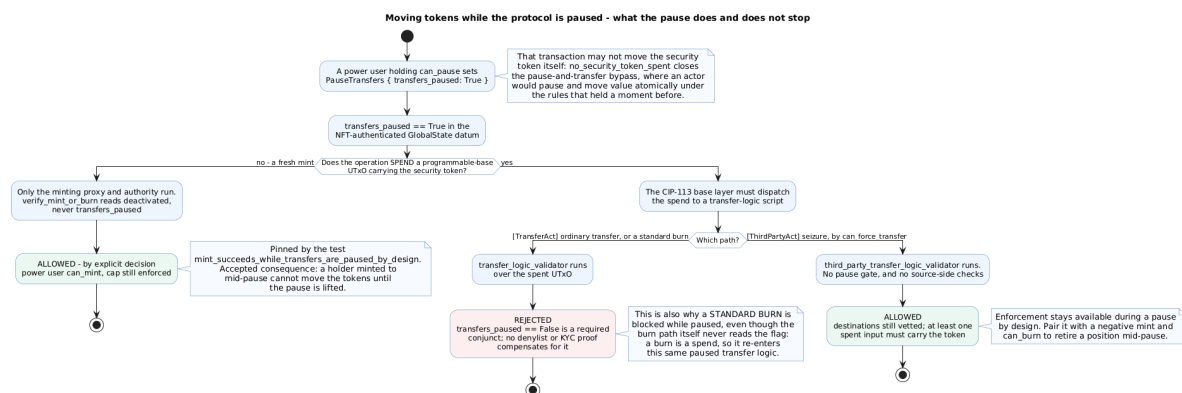




Sanction blocks **both directions** — the same absence proof is demanded of every destination, so a denylisted address can neither send nor receive. That matches CMTAT, whose `ValidationModule` checks `from` and `to` alike. And because the check keys on the bare hash, sanctioning a hash sanctions both its verification-key and script credential forms. The only route by which a sanctioned position moves is `third_party_transfer_logic_validator`, the seizure path, which runs no source-side checks and requires a power user holding `can_force_transfer` (ID 17).

Denied transfer: a paused protocol

The pause is a single flag in the authenticated `GlobalState` datum, and `transfer_logic_validator` requires it to be `False`. An operation is stopped only if it **spends a programmable-base UTxO**, because only then does the CIP-113 base layer dispatch to the transfer logic at all. What the operation is called has nothing to do with it.



That single rule explains all three outcomes in the [Implementation Details](#) table. A **mint** spends no such UTxO, so the transfer logic never runs and minting stays available — a deliberate divergence, pinned by a test. A **standard burn** does spend one, so it re-enters the paused transfer logic and is blocked in practice, even though nothing on the burn path reads the pause flag. **Seizure** is dispatched to `third_party_transfer_logic_validator`, which has no pause

gate, so enforcement stays available; pairing it with a negative mint is the only way to retire a position mid-pause, and needs `can_force_transfer` and `can_burn`.

Lifting the pause is `PauseTransfers { transfers_paused: False }` by a `can_pause` power user — unless the protocol has been deactivated, in which case the terminal guard rejects every `GlobalState` spend and the pause becomes permanent (ID 14).

Frequently Asked Questions

Q: What is a "validator" here?

A Plutus script — never a stake pool, a block producer, or any party that approves a transaction. It is a pure predicate over the single transaction being submitted, and its hash is its identity: for a spend the hash is the address, for a mint the policy id, for a withdrawal the stake credential. [What "validator" means here](#) covers the script purposes and lists the ten this deployment compiles.

Q: What is a "node" here?

Three different things wear the word in this document, and only the first two appear in this codebase.

- **A list node.** eUTXO has no mapping type, so a set is built as a sorted linked list whose elements are UTxOs. Each such UTxO is a node: it sits at the list's spend address, carries exactly ADA plus one NFT of the list's policy, and its asset name is `"Node"` followed by the 28-byte key. The **key is minted into the asset name** and is immutable; the **link to the next key, and any payload, sit in the inline datum** and can be rewritten by whoever holds the UTxO. That split is why every reader also checks the node still lives at the list's address.
- **A CIP-113 registry node.** Structurally the same idea, but an element of the base layer's registry rather than of either list here. It names the minting-logic and transfer-logic scripts for a policy. This is the sense in play in `UpgradeRegistryNode` and in "no registry node spent during a supply change".
- **A `cardano-node`.** The network client. It never appears in this document.

The two lists in this deployment differ only in payload. A **denylist** node's datum is inspected by nothing — presence in the list *is* the sanction. A **power-users** node's datum carries the five role flags, so there the payload is the authority, which is what makes the address pin load-bearing: a node NFT carried into a private wallet would otherwise keep granting whatever flags its holder wrote.

Q: Why must every transfer supply a "covering-node reference input"?

Because a validator cannot look anything up. It sees only the transaction in front of it, so it cannot ask whether a party is on the denylist — the transaction builder has to supply the evidence and the validator only checks it.

Since the list is sorted, absence is witnessed by a single node: the one whose key sits immediately below the party and whose link points immediately above them. Nothing can hide in that gap.

`covers_key` requires `covering_key < target < covering_link`, with **both comparisons strict**, treating the root as $-\infty$ and a `None` link as $+\infty$.

Reference input means read without spending, so a transfer never consumes the denylist and many transfers can cite the same node in the same block.

Q: What happens when the party is actually sanctioned?

No covering node exists, so the transaction cannot be built. If the party's key is `0x77...`, the node below it links to `0x77...` and fails `target < link`; the party's own node fails `key < target`; any node further down links to its own successor, at or below the target, and fails the upper bound too. Both edge cases are pinned by tests (`covers_target_equal_key_sad`, `covers_target_equal_link_sad`).

Nothing in the code detects the sanction and rejects it. The proof the transaction needs simply cannot be constructed. Forging one fails on three counts: the node must carry the list NFT with a well-formed datum, it must still sit at the denylist spend address, and a missing or wrong reference index traps rather than failing quietly. The [Annex](#) traces the full path.

Q: Why is a zero-ADA withdrawal part of every transfer?

A spending validator runs once per UTxO spent, so a transfer consuming ten of a holder's UTxOs would repeat the whole compliance check ten times — and each run would see only its own input, which is useless when the rule is about the *set* of parties in the transaction.

A withdrawal of zero balances trivially and moves no value, but it makes the ledger invoke the named script **exactly once, with the whole transaction in view**. Under Plutus phase-2 validation a script named as a withdrawal key must execute and succeed or the transaction fails, so the mere presence of that key proves the script ran and approved this transaction. That is what lets the minting proxy delegate every real rule to a swappable authority, and it is why the proxy uses `must_run_script_withdrawal` rather than `must_be_signed_by_credential` — a signature would prove consent, not that any logic executed.

The cost is that the stake credential must stay registered, which is why all four withdraw-0 validators accept `RegisterCredential` and refuse every other certificate.

Reference

Implementation being approved:

Component	Repository	Version / Commit
Contracts (assessed)	https://github.com/cardano-foundation/cp-t-rwa-ch-de-cmta-reference	commit <code>ff5624e</code> (2026-08-24)
Aiken package	<code>ft/rwa</code>	v0.0.0; Aiken <code>v1.1.23</code> ; Plutus \
CIP-113 specification	https://github.com/HarmonicLabs/CIPs/blob/master/CIP-0113/README.md	Nuzzi, Coppola, Gargiulo, Di Sai
CIP-113 base layer	(deployed base layer)	guarantees verified against con <code>018415d</code>
Predecessor (assessed separately)	https://github.com/cardano-foundation/cip-113-programmable-tokens-platform	<code>security-token</code> substandard, <code>bab6fc8</code> — see README-cip113-security-token.md
Upstream origin (assessed separately)	https://github.com/FluidTokens/fn-bafin-cardano-sc	commit <code>67ab7d9</code> — see README-bafin-cardano-sc.md . The CF substandard's <code>aiken.toml</code> na origin as <code>easystaking-com/fr</code> <code>cardano-sc</code> (unverified path).

Component	Repository	Version / Commit
Penetration tests	documents/pentesting/	FT Labs, 23 and 26 June 2026 (commit <code>dd2b754</code> , <code>1beeed6</code>) — do not assessed commit
Internal security review	documents/security/security-fixes.md	11 defects (2 critical), fixed and regression-pinned

Assessment template submodules (unchanged from the template — the CMTAT Solidity reference this Cardano implementation is compared against):

Submodule	Repository	Version	Commit
CMTAT	https://github.com/CM-TA/CMTAT	v3.2.0	49544f4de1993008acfc9e848d0bf03bd31d8579
SnapshotEngine	https://github.com/CM-TA/SnapshotEngine	v0.3.0-1-g19e0b56	19e0b569bf5823aa8cec5760f080a932a9ac940e
RuleEngine	https://github.com/CM-TA/RuleEngine	v3.0.0-rc2-2-g9c0aa70	9c0aa70aae08047e4062beab0f89f92bd60252c0
Rules	https://github.com/CM-TA/Rules	v0.3.0	91c21c1191e84ff938892267ec443b0d1bb9efb0 docs: explain what a validator is in the assessment annex